Best Practices for Machine-Readable Data and Code Sharing in Psychology

Daniël Lakens[1], Cristian Mesquida[2], and Lisa DeBruine[3]

[1]Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

[2]Industrial Engineering and Innovation Sciences, Eindhoven University of Technology

[3]Institute of Neuroscience and Psychology, University of Glasgow

Author Note

Daniël Lakens https://orcid.org/0000-0002-8393-5316

Cristian Mesquida https://orcid.org/0000-0002-1542-8355

Lisa DeBruine https://orcid.org/0000-0002-7523-5539

A reproducible version of this manuscript available at on GitHub (https://github.com/scienceverse/machine_readable_FAIR_data_and_code). The authors have no conflict of interest to declare. This work was funded with the support of the Netherlands Organization for Scientific Research (NWO Vici grant VI.C.241.013). Author roles were classified using the Contributor Role Taxonomy (CRediT; https://credit.niso.org/) as follows: Daniël Lakens: conceptualization, writing, funding acquisition; Cristian Mesquida: conceptualization, writing; Lisa DeBruine: conceptualization, writing

Correspondence concerning this article should be addressed to Daniël Lakens, Industrial Engineering and Innovation Sciences, Eindhoven University of Technology, Den Dolech 2, 5600 MB, Eindhoven, Email: D.Lakens@tue.nl

Abstract

The FAIR principles state that research materials should be Findable, Accessible, Interoperable, and Reusable (Wilkinson et al., 2016). Adhering to these principles requires data and code to be organised in consistently structured repositories and accompanied by standardised, machine-readable metadata. However, implementing these principles can be complex and effortful. Previous literature has disussed how to make research materials FAIR, with particular focus on data and metadata. Here, we first describe best practices and existing standards for repository organisation, content, and file naming. We then demonstrate how researchers can use Metacheck to identify issues that hinder the FAIRness and machine-readability of repositories and their contents.

Keywords: FAIR, Data sharing, Reproducibility, Open Science

Best Practices for Machine-Readable Data and Code Sharing in Psychology

Access to the data and code that underlie findings in the scientific literature allows peers to scrutinize the evidential basis for empirical claims, and to increase the efficiency of knowledge generation through re-use. The European Code of Conduct for Research Integrity states: "Researchers, research institutions, and organisations ensure that access to data is as open as possible, as closed as necessary, and where appropriate in line with the FAIR Principles (Findable, Accessible, Interoperable and Reusable) for data management." (ALLEA, 2023). Similarly, the Dutch Code of Conduct for Research Integrity states that: "Ensure that data, software, and research materials are as open and accessible as possible in accordance with the FAIR principles." (NWO, 2026). Accordingly, research materials should be managed in accordance with the FAIR Principles (Wilkinson et al., 2016). Meeting these standards requires repositories to preserve the information needed to locate, understand, and use the deposited research outputs. To this purpose, Data Management Plans (DMPs) help researchers specify how data will be managed, documented, preserved, and shared throughout a project (Michener, 2015; Miksa et al., 2019). Our recommendations complement this planning by providing practical guidance on how to chose a repository and how to organize and document the different types of files within a research repository to support implementation of the FAIR principles.

Following a data management plan, making research files available and including an appropriate license does not, by itself, ensure that others will be able to use them. In psychology, Hardwicke et al. (2021) examined 25 articles awarded open-data badges between 2014 and 2016 and reproduced the target results. For 64% of the articles there were numerical discrepancies when trying to reproduce the results, after contacting the authors 37 out of 789 checked values remained unmatched. The principal obstacle was the unclear reporting of analytical procedures. Later, Trisovic et al. (2022) examined more than 9,000 R files from replication datasets deposited in Harvard Dataverse and 74% failed to compile without errors during initial execution. They also report how it is essential to use descriptive file naming, and found that about one third of the files (32%) contained a space in the name, a highly discouraged practice. These studies show how code and data sharing is not enough to

guarantee reuse. Such barriers prevent the ability to properly inspect materials, adapt code, or reuse the data for a new research question.

The FAIR Principles provide a framework for improving the Findability, Accessibility, Interoperability, and Reusability of research outputs by both people and machines (Wilkinson et al., 2016). Their emphasis extends beyond whether software can read a file: automated systems also need information about what a resource contains, how it relates to other resources, and under which conditions it can be used. Those Principles were explicitly developed with both human and machine users in mind. Although it is important that individual can find, access, and re-use data, the benefits of FAIR data and code can only be realised when scientific resources are also accessible and processable by machines. Therefore, research data and code should be shared in machine-readable formats (e.g., see Lakens & DeBruine, 2021 for an example of the benefits of machine-readable hypothesis). Machine-readable FAIR data and code will make it possible to retrieve and integrate data into large, searchable databases, and facilitate the automation of tasks, such as checking data and code for adherence to best research practices. Applying FAIR to research software introduces additional considerations, including software versions, dependencies, and the relationships among software components. These considerations motivated the development of the FAIR Principles for Research Software, or FAIR4RS (Barker et al., 2022; Chue Hong et al., 2022; Lamprecht et al., 2020). This guide summarises best practices for sharing data and code in ways that make them findable, accessible, interoperable, and reusable by both humans and machines.

A machine can only find data and code reliably and automatically if they have a stable digital identifier and metadata. Metadata are structured information that describe other data or files, providing details such as their content, structure, format, origin, and how they should be interpreted. In addition, data and code should be accessisable through standardized protocols (e.g., HTTPS or an Application Programming Interface (API)); provided in formats that can be used by other software (e.g., non-proprietary formats accompanied by metadata describing how files should be read in); and re-usable (e.g., by providing a codebook and a license).

When data and code are shared this way, an automated computational system can identify what files contain — including both their format and purpose — based on metadata or

filenames; determine whether they can be accessed and reused, for example given the license or privacy regulations; and process the data and code accordingly. Ideally, each of these steps should be unambiguous and deterministic, minimizing the need for probabilistic inference by machine-learning algorithms or large language models. This guide is intended for researchers and data managers. It discusses best research practices for making data and code machine-readable and FAIR, including where to share data and code, repository organization, file naming, file formats, codebooks, README and requirement files, data layout, column naming, codebook generation, specification of scales and measures and adherence to existing schemas and standards.

Existing recommendations address different aspects of research sharing. Some focus preparing and documenting code for reuse(Abdill et al., 2024; Eglen et al., 2017), while others address the organisation of data and code, emphasising structured files, informative metadata, reusable formats, and explicit licences(Jenkins et al., 2023). More recently, TADA has proposed minimum requirements for sharing functional analytical code (Ivimey-Cook et al., 2025), and the SORTEE guidelines have established a staged framework for assessing archived data and code, from their availability and documentation to their correspondence with a manuscript and computational reproducibility (Pick et al., 2026).

The proposed work brings these recommendations together at the level of a complete research repository in psychology. A repository may contain data and code, codebooks, scripts, questionnaires, stimuli, generated outputs, and supporting documentation. Reusing these resources requires their purposes and relationships to be clear: which codebook describes a dataset, which script produces an output, which materials belong to a particular study, and which licence applies to each resource. A standardised repository structure, supported by explicit and consistent metadata, can make these relationships easier to establish without requiring users to reconstruct them from the manuscript or contact the original authors.

This guide synthesises existing standards and practical recommendations into actionable guidance organised around repository structure, contents and file types. It covers repository selection, folder structure and naming conventions, data formats and codebooks, code, outputs, research materials, licences, and approaches to assessing FAIRness. Its

emphasis on machine-readable descriptions builds on broader efforts to make research outputs easier to discover and reuse automatically(Lakens & DeBruine, 2021). Ultimately, this work aims to support the creation of repositories in which peers and software can identify the available resources, understand how they relate to one another, retrieve relevant files, and determine the conditions for their reuse.

1 Repository Choice

There are many data repositories, each with different strengths and limitations. When choosing a repository, several features are worth considering. These include whether it allows repositories or files to be shared anonimously, whether access to data can be restricted to specific users, and whether it provides persistent identifiers (e.g., DOIs), stable versions, version control, appropriate metadata, and programmatic access. Other important considerations include compatibility with relevant regulations, such as the General Data Protection Regulation (GDPR), compliance with funder requirements, and restrictions on the types or quantities of data that can be archived.

Curated registries, such as re3data (Pampel et al., 2023) and OpenDOAR, can help researchers identify suitable repositories based on specific characteristics. This section discusses a selection of crepositories commonly used in psychology, including Zenodo, GitHub, the Open Science Framework, ResearchBox, Figshare, and PsychArchives. Researchers may also have access to dedicated research data repositories provided by they country or university, such as the UK Data Service or DataverseNL.

It is important to distinguish between platforms designed primarly for code development and collaboration and those intended for archiving research outputs. GitHub is primarily a platform for developing and collaborating on code rather than a data repository. However, public GitHub releases can be archived through Zenodo, which assigns a DOI to each release and preserves a citable snapshot of the repository. Zenodo also provides a concept DOI that resolves to the record containing all versions, alongside a version-specific DOI for each release.

Table 1 provides a brief comparison of repositories commonly used in psychology and related social sciences. The table is intended as an overview rather than a ranking, as the

features of institutional services based on platforms such as Dataverse or Figshare may differ from those of their general-purpose counterparts. Repository features and policies may also change over time, so researchers should consult the current terms and technical documentation of the specific service before depositing data. The comparison is based on dimensions discussed in a recent overview of repository choice, including persistence, machine access, metadata, sharing for peer review, usability, and data protection.

The suitability of a repository also depends on whether it supports machine access. Public APIs allow automated systems to retrieve metadata and files without relying on manual interaction with a web interface. ResearchBox now provides a documented API; PsychArchives has an API, although its documentation and programmatic access to some restricted files remain limitations. These features should be checked against the intended use of the shared materials.

Repositories that support anonymous or restricted-access sharing can be particularly useful during peer review. Anonymous links are available through PsychArchives, Dataverse, ResearchBox, and OSF; Zenodo and GitHub can also be used for review sharing, although this may take additional steps. Researchers should check what information a reviewer link exposes and whether the files themselves have been checked for identifying information.

For sharing anonymized data and research materials, Zenodo may be a suitable choice because it provides persistent DOIs, versioning, and options for restricting file access when necessary, including for materials under review or data that cannot be made publicly available. However, restricted access alone does not establish that a repository is appropriate for identifiable or sensitive personal data. Researchers should check applicable institutional, ethical, and data-protection requirements, particularly for pseudonymized data, before depositing it.

When choosing a data repository, therefore, do not simply ask whether it allows you to upload your files. Consider whether its infrastructure supports the level of openness, persistence, accessibility, and machine readability required for the research materials to adhere to FAIR principles.

2 Repository content and organization

Repositories should contain not only data but also materials that support replication and reuse. These may include code used to process data or run analyses, and study materials such as stimuli, questionnaires, and other experimental materials. These file types serve different purposes and may be reused in different ways. Study materials vary across projects and are not always machine-processable, whereas data and code often contain structured information that people and software can access and process. This can support activities such as living meta-analyses or searches for studies that used a particular questionnaire. Repository metadata describe the project and its files, while codebooks explain variables and values in data files; both support interpretation and reuse.

2.1 Overall structure and file/folder naming conventions

Naming conventions are not universal, so folder structures and names vary across projects and research fields. A few shared principles can nevertheless make repositories easier for people to navigate and for software to process.

2.1.1 Folder structure. Folders should group related files. A research repository can include folders named `data`, `code`, `output`, `materials`, and `documentation`, with an `other` folder for files that do not fit these categories. The top-level folder corresponds to the paper and contains files that apply to the repository as a whole. A README is one example: it describes the repository contents and gives instructions for reuse in a human-readable form.

Psych-DS, a data structure for social science research, recommends a subfolder for each project in which data were collected. For a multi-study paper, this means creating a folder for each study (e.g., `pilot-classification`, `study-mood-classification`, and `study-moderator-effect-mood`). Choose clear, consistent study labels early, for example when preparing an ethics application or preregistration. Because studies may be reported in a different order from the order in which they were conducted, and numbers alone do not describe a study, avoid naming folders only `study-1`, `study-2`, and so on. Short descriptive labels help connect repository files to the corresponding study in the manuscript. Once the order of studies in the paper is settled, numbers can be added to the folder names (e.g., `01-pilot-classification`, `02-study-mood-classification`).

At a minimum, each study folder should contain a `data` folder, and each data file should be accompanied by a codebook.

When data are analysed using individual code files, place them in a `code` folder. A study folder may also contain an `output` folder for results generated by analysis scripts. Repositories may include a `documentation` folder for files such as the preregistration and manuscript, and a `materials` folder for stimuli, survey files, and other study materials.

Software may search for these folders using a limited set of aliases. For example, a documentation folder might be called `reports` or `docs`, and a materials folder might be called `assets` or `stimuli`. If the range of names is limited, software can use a predefined list to identify folders containing particular types of files. This reduces reliance on large language models, which are less reliable and more computationally expensive than rule-based classification for this task.

```
repository/
  README.md
  code/
      data_cleaning.R
      analysis.R
      simulation.R
  data/
      raw_data.csv
      processed_data.csv
      codebook.csv
  output/
      results_simulation.csv
      table_results.csv
      figure_results.png
  materials/
      survey.txt
  documentation/
      preregistration.txt
  renv.lock
  LICENSE
```

2.1.2 File and folder naming conventions. Files and folders should be named so people can understand them and software can classify them. Because files may need to be identified by several properties (e.g., study identifier and file type), use a consistent naming pattern that makes these properties distinguishable. Underscores can separate properties:

`study-01-classification_data.csv`, for example, identifies both the study (`study-01`) and the file type (`data`). For classification, using underscores or hyphens matters less than consistently including informative category terms. A CamelCase name such as `codePowerSimulationStudy1` can also be classified if it contains a recognizable category term.

Each filename should include a term that helps software assign it to a category. Common categories are `data`, `code`, `output`, `documentation`, and `materials`. Include one of these terms, or a recognised synonym such as `data_dictionary` or `stimuli`, together with a descriptive keyword for the file's specific contents (e.g., `preprint_documentation.pdf`). Machines can classify files using filename and extension rules, with a large language model as a fallback for files that remain ambiguous.

Be consistent with capitalization because some software treats `Data.csv` and `data.csv` as different files. Lowercase names are generally preferable unless a schema or data standard requires capitalization. Avoid spaces and special characters. Use leading zeros so numbered files sort in the intended order (e.g., `02-data` before `11-data`). Choose enough digits for the largest expected sequence. Dates should use the ISO 8601 format `YYYYMMDD` (e.g., `20260724_data.csv`); the extended format `YYYY-MM-DD` may be used when required by a particular convention.

Long folder names can produce file paths that exceed limits imposed by file systems or software. Some systems limit filenames to 255 characters, and downloading a repository adds the local folder path to each file path. For example, if a repository is downloaded to `C:\Users\dlakens\Downloads`, the paths within the repository should remain below 228 characters. As a practical guide, keep directory paths below 100 characters and individual filenames below 50 characters, leaving room for the local path.

Names should match the terminology and identifiers used in the manuscript. If the paper refers to `Study 1`, for example, use the same study identifier in the corresponding folder rather than an unrelated label such as `Survey 1` or `Aggression Study`. This helps software link repository files to the relevant part of a multi-study paper. Metadata, including RO-Crate metadata discussed above, can describe file and folder contents that cannot be conveyed

reliably through names alone.

2.1.3 README and repository-level metadata. Every repository should include a README. It helps people browsing the repository understand its contents and provides instructions for reuse. Because free-form README text is less reliable for automated processing, it should be complemented by structured metadata containing information that cannot easily be retrieved from the files themselves and is useful to re-users.

Structured metadata should describe the repository and its files, including information that is difficult to retrieve from the files themselves. It should identify the research project and its creators. Because machines may find a repository independently of the article, describe the project separately from the manuscript and include the manuscript DOI when one exists. Adding file descriptions to a JSON metadata file can also help check whether file names are unambiguous and provide a reliable description when they are not.

Repository metadata can be challenging because different standards operate at different levels. Dublin Core provides generic metadata about a repository, such as its title, description, creator, and license. The DDI Alliance provides ISO-standard specifications for describing research data in the social, demographic, economic, and health sciences. DDI-Lifecycle 3.3 can describe study design, sampling, and data to support reuse.

Field-specific guidelines are needed to clarify which metadata should be shared. Tools should make it easy to generate structured JSON metadata that follows widely used specifications. Such guidance could draw on existing standards, including DDI-Lifecycle 3.3 for study, sample, and data-handling descriptions; CRediT for contributor roles; Schema.org for bibliographic metadata; and the RO-Crate specification (Soiland-Reyes et al., 2022) for describing relationships among repository files and supporting machine-readable FAIR data and code.

Some information is difficult to retrieve from a paper but valuable for reuse, and can be recorded directly in metadata. Research fields should determine which information is important enough to include. In many empirical disciplines, sample size is useful for meta-analysis and helps indicate the strength of the evidence. DDI-Lifecycle provides standardized fields for sample eligibility, inclusion and exclusion criteria, recruitment, target

sample size, completed responses, and response rate.

Metadata can also describe data provenance: how and under what circumstances the data were collected and handled. Useful details may include the collection period, whether sessions were supervised or conducted individually or in groups, whether collection took place during an exam period, and whether data were collected online or in person. DDI's `LifecycleEvent` schema allows researchers to record data-handling steps, including what happened, when, and who was responsible. This can document contributions by research assistants, technicians, and data managers who are not authors, making their work visible and clarifying responsibility for each processing step. A schema for open science practices could also indicate whether shareable data, code, and materials have been made available and whether the study was preregistered.

Researchers may also share documentation such as codebooks, preregistration documents, the manuscript, supplementary materials, and README or metadata files. These documents provide context for interpreting and reusing data and code, so identify them clearly in the repository and will be discussed in detail in Section 2.2.4.

2.1.4 Repository size, packaging, and selective downloads. Online repositories often provide generous storage limits. For example, Zenodo has a standard limit of 100 files and 50 GB per record; additional storage can be allocated (file limits; storage quota). Large repositories can still make reuse difficult: users may lack the time or storage to download files they do not need, and automated tools may impose limits on the size of individual files or entire repositories. Organise and label files so users can download only the materials relevant to their purpose. For example, someone looking for analysis data should be able to retrieve the data and code without also downloading large video files used as stimuli.

Compression can reduce the size of large files. For multiple files, a `.zip` archive can be useful: its central directory lists the files it contains, including their names and sizes. Because this directory is near the end of the archive, software can retrieve it using HTTP range requests—when the server supports them—without downloading the entire archive. This allows users to check whether an archive contains relevant files before downloading it in full. Prefer ZIP to proprietary formats such as `.rar` when interoperability matters.

2.2 Data

2.2.1 Raw and processed data. Where possible and appropriate, researchers should share raw data: the data in the form in which they were originally generated or collected. Examples include files exported from Qualtrics, audio recordings, and individual-participant EEG data. Raw data provide a record of the original observations and allow others to understand and reproduce the processing steps applied to them.

Sharing raw data is not always possible or appropriate. The data may not have been collected digitally, may contain information that cannot be adequately anonymised, or may be subject to ethical or legal restrictions. In such cases, share the earliest digital representation that can be made available, such as scanned surveys or anonymised transcripts. When data must be processed before they can be analysed or shared—for example, to remove identifying columns or convert unstructured data into a table—use code to perform these transformations whenever possible. Share the resulting minimally processed data together with the code used to create it. The README should explain how the processed data were created from the raw data and describe the transformations applied.

Using code rather than manually editing raw data has two advantages. It creates an auditable record of the transformations, allowing others to inspect the workflow for potential errors, and it makes the transformations reproducible: the same code can be rerun if errors are found, processing needs to be updated, or a new data version is required. Avoid manual changes to raw data; document any that cannot be avoided.

2.2.2 File formats and data export. Store data in open, machine-readable formats such as `.csv`, or in widely used software-specific formats such as `.sav`, `.jasp`, or `.omv`. For very large files, consider whether the format and repository support partial downloads through HTTP range requests. CSV files can allow parts of a file to be retrieved without downloading it in full, whereas formats such as `.xlsx` may require downloading the whole file.

When data are collected using commercial software, or on paper (e.g., a printed questionnaire), the raw data may not be available in an open format. In such cases, store the original files in a folder named `raw` inside the `data` folder, preferably in a `.zip` archive if there are many or large files. Also export the data, as directly as possible, to an open format such as

CSV. CSV files should follow the RFC 4180 specification and use UTF-8 encoding. Enclose commas within data values in double quotation marks (e.g., `pp1, 34, "To be, or not to be."`). Avoid plain `.txt` files for tabular data unless their structure is documented, because the extension alone does not tell software how to interpret their contents. Similarly, use `.rds` files for individual R objects rather than storing them in `.RData` files, which can contain multiple objects and make their contents harder for software to identify. File extensions should match the underlying format; for example, save a JSON file as `.json`, not `.csv` or `.txt`.

Compression can substantially reduce the size of large data files. Gzip (`.gz`) is particularly useful for large CSV files because compressed CSV files can be read directly without first being decompressed. We recommend gzip compression for CSV files larger than approximately 10 MB.

Spreadsheets (`.xls` or `.xlsx`) can also be used to store data. While the `.xlsx` format is based on Office Open XML and widely supported by open-source software, the `.xls` format relies on Microsoft's proprietary format and should be avoided to prioritize open-source alternatives. Do not store separate datasets on multiple tabs, because filenames cannot identify the contents of individual tabs; save each dataset as a separate file. Merged cells, empty rows or columns, and other layouts that depart from a rectangular table can also make spreadsheets difficult to import into data-analysis software.

Some analysis software, such as JASP and jamovi, can store data, codebook information, analyses, and output in a single file (`.jasp` or `.omv`). When all analyses can be conducted in one of these programs, sharing the resulting file may provide a simple way to make the data, labels, analyses, and output available together. This can require less setup than sharing separate data and code files, particularly for researchers who do not use a scripted workflow.

2.2.3 Tabular data files. Leave cells for missing values blank when the software used to read the data can interpret them as missing (e.g., as `NA` in R). Avoid ad hoc missing-value codes such as `999 = no response` or `888 = hardware failure`. If such codes must be retained—for example, because they are present in the raw data—document them in the metadata or codebook. Do not rely on colour, comments, or hyperlinks to convey information,

as these may be lost when data are imported into other software; use dedicated columns instead (e.g., a `notes` column). CSV files and spreadsheets should have a single header row in the first row of the file.

Data files should be rectangular tables with a tidy structure: one header row, one variable per column, and one observation per row. Software such as SPSS, JASP, and jamovi expects this structure. Spreadsheet editing can introduce problems such as headers spanning multiple rows, empty rows or columns, and notes placed below or alongside the data.

Column names should follow more restrictive conventions than filenames. Use only letters, digits, and underscores (e.g., `participant_id`, `mean_reaction_time`) to make names easier to reference in analysis code; in R, for example, a column can be accessed as `df$participant_id`. Names containing spaces or dashes must be quoted or accessed using another explicit syntax (e.g., `df[["participant id"]]`). Special characters, including diacritics, may need to be escaped or may be renamed when data are imported into other software, sometimes creating duplicate names. Keep names shorter than 32 characters, the limit in some analysis packages. Names must be unique within each file, consistent for the same variables across files, and identical to the corresponding names in the codebook.

Use descriptive column names that can be understood without consulting the analysis code (e.g., `self_esteem_scale_1` rather than `SE1`; avoid generic names such as `VAR1`). Where possible, use terminology that matches the manuscript. For items belonging to the same scale or construct, use a consistent prefix and sequential numbers (e.g., `self_esteem_scale_1`, `self_esteem_scale_2`, `self_esteem_scale_3`). This convention can also identify subscales (e.g., `panas_positive_1` for the first item of the Positive and Negative Affect Schedule's positive-affect subscale), making it easier for software to identify scales and subscales. Place items from the same scale in adjacent columns where possible. When several columns describe the same response, such as the response and its reaction time, make each column's meaning clear (e.g., `mood_1_response`, `mood_1_reaction_time`).

Avoid rows or columns containing only empty values, which may indicate that a value was not recorded, a column was deleted, or a row or column was accidentally left blank. Remove empty rows or columns where possible. If an empty variable must be retained,

document why it is present rather than assigning an arbitrary value such as `0` or `none`. Column names and values should not contain leading or trailing whitespace (e.g., use `yes`, not `yes`). Avoid mixing text and numbers in otherwise numeric columns: a value such as `no response` can cause software to interpret the entire column as text. Use a consistent missing-value convention and document it in the codebook. Before sharing data publicly, check for personal information such as email addresses or IP addresses and remove or anonymise it as appropriate. Screen open-text responses for identifying information as well.

2.2.4 Metadata and codebooks. Field-specific metadata standards can describe files commonly used in particular research areas, such as BIDS for brain data and Psych-DS for psychological and behavioral research data.

In addition to repository-level metadata, individual studies can be organized in Psych-DS-compliant subfolders. Each Psych-DS dataset requires a `dataset_description.json` file containing a machine-readable description of the dataset. Variable-level information, such as definitions and measurement details, can be provided in a data dictionary or related metadata. Unlike `ro-crate-metadata.json`, which describes provenance, contributions, and general relationships among files, these data descriptions help explain what the variables represent and support their reuse. (Psych-DS data dictionaries)

Machine-readable data descriptions support findability and reuse, and allow users to search for studies that collected particular types of data. Psych-DS uses Schema.org's Dataset model with additional Psych-DS fields. Structured descriptions can be generated automatically from well-organized repositories when the codebook is accurate and file and folder names are informative.

A codebook, or data dictionary, describes the contents of data files; the terms are often used interchangeably. Buchanan et al. (2021) note that a codebook traditionally describes survey variables and may be created during data collection, whereas a data dictionary is often created during planning and may also describe the structure of data files. Regardless of the term used, each data file—or each set of files with the same structure—should have an accompanying codebook. Data files should have a single-row header, and the codebook should explain what the column names and values mean. Some formats (e.g., SPSS, Stata, SAS, JASP,

jamovi, and Qualtrics) support variable and value labels, allowing some codebook information to be embedded in the data file. If a standalone codebook is needed, software packages can assist with its creation (see for example Arslan, 2019). A complete codebook should contain six types of information:

1. **Column name:** Use the name exactly as it appears in the data file (e.g., `panas_1`).
2. **Label:** Provide a plain-language description (e.g., `positive and negative affect schedule item 1`). For variables not derived from survey questions, describe what the values represent, such as reaction times or correct/incorrect responses. If a variable belongs to an established scale, include the scale's full name. At a minimum, provide the column name and label.
3. **Value labels:** For variables with a limited number of response options, explain what each value means (e.g., `1 = very negative; 2 = somewhat negative; 3 = neutral; 4 = somewhat positive; 5 = very positive`). If only the endpoints are labelled, document those (e.g., `1 = very negative; 5 = very positive`).
4. **Question or item text:** For survey data, include the verbatim question or item shown to participants.
5. **Missing-value codes:** If missing values are coded differently across columns, specify the code used for each variable.
6. **Coding instructions:** Describe any processing needed to interpret or calculate scores, such as reverse-scoring an item or calculating a mean from PANAS items 1–10. If instructions are too complex for a table, explain them in a README or provide code that transforms the data.

One way to create a codebook is to enter the relevant information in data-analysis software. In SPSS (`.sav`), Stata (`.dta`), SAS (`.sas7bdat`), JASP (`.jasp`), jamovi (`.omv`), and Qualtrics (`.qsf`), variable and value labels can be stored with the data. These formats can therefore combine data and some codebook information in one file, but may not capture every field needed for a complete codebook. For example, not all formats provide fields for documenting missing-value codes or the exact survey-item text, and none provides a dedicated

field for coding instructions. Provide a separate codebook when information is missing from the data file.

Psych-DS provides a schema for describing data files and variables in JSON. Codebooks can also be tabular, with a column for each type of information recorded about a variable. An example is provided in Table 2.

Tabular codebooks can be stored as `.csv` files, but a spreadsheet may be easier to create. Copy the column names from the data file and transpose them (i.e., convert the header row into a column) in the codebook so that names match exactly. Store values as text where needed, because Excel may convert values such as `1-5` into dates such as `5-Jan`.

Store codebooks as `.ods`, `.xlsx`, or `.csv` files. The first two formats are generally easier for people to read, while CSV is more interoperable. Avoid Word or PDF files because their contents cannot be reliably parsed by machines. Name a codebook `codebook` or `data-dictionary`. If a folder contains one data file, its location identifies the corresponding codebook; if it contains several, include the data filename in each codebook name (e.g., `pilot-1a_data.csv` and `pilot-1a_codebook.csv`).

2.3 Code

Code files are part of the research record and should be organized so that their purpose and relationship to other repository files are clear. These may include scripts, notebooks, and configuration files created or adapted for the project. Brief documentation should identify what each file does and which data or outputs it relates to, helping others understand and reuse the code.

When a script contains multiple analyses, results should be organised so that users can identify which analysis produced each output. One approach is to use separate scripts and corresponding output files with consistent naming conventions (e.g., 01_simulation.R and 01_results_simulation.csv), and store the resulting files in the output folder. The execution of these scripts can be automated using the {targets} R package, which tracks dependencies between scripts and functions to determine the order in which they should run. This makes it easier to manage projects with multiple interdependent scripts and ensures that analyses are executed in the correct sequence.

The practice of clearly labeling files in repositories is also important to distinguish links to repositories that contain existing software used in research (e.g., a GitHub link to a software tool created by others that was used to process physiological data) from links to repositories that contain code generated by the researchers themselves for their research (e.g., code for an agent-based model that researchers have created as part of their research).

Computational reproducibility refers to the ability to produce the same outputs from the same data and code inputs. However, differences in software versions, package dependcies or computational settings can prevent analyses from being reporduced. To address this, the software and dependencies required to run the analysis should be clearly specified. For example, R projects can include an renv.lock file and Python projects a requirements.txt or environment.yml file. For analyses that depend on system libraries or software beyond R or Python packages, containerized solutions such as Docker, Nix, or Code Ocean provide stronger guarantees that the computational environment can be reconstructed in the future.

2.4 Output

Researchers may also share output files such as statistical results, tables, figures, and simulation results. These files allow peers to inspect research outputs directly, without necessarily having to rerun the analyses. These outputs are particularly useful when generating them requires substantial computation or when they help peers verify reported results. In these cases, analytical results should be saved in reusable formats (e.g., .rds or .csv files) rather than being available on the software console. For example, in R, researchers can store key analytical results in objects and save them for later inspection or reuse. Alternatively, when analyses require little computational time, workflows such as Quarto allow analytical results to be embedded directly in rendered documents, reducing the need to store outputs separately.

2.5 Materials

Study materials may include questionnaires, rating scales, stimuli, survey files, and other resources used to conduct a study. Field-specific metadata standards can make structured materials, especially scales and questionnaires, easier to identify and reuse. The Open Scales Database, for example, provides a specification for scales in psychology.

The Open Scale Definition (OSD) is a machine-readable format for describing

questionnaires and rating instruments. It is part of the Psychology Experiment Building Language, which was created for building and running psychological experiments (Mueller & Piper, 2014). An `.osd` file has two top-level parts: `definition`, which describes the scale's structure in a language-neutral way, and `translations`, which stores the scale text by language code so that the same structure can represent different language versions. The OSD specification includes `scale_info`, `likert_options`, `dimensions`, `items`, and `scoring`; an entry can be set to `NULL`. The `scale_info` field includes the scale's name, code, abbreviation, original publication citation, license, and other descriptive information. The remaining fields describe response options, dimensions and subscales, scoring, and item text in available translations. Machine-readable scale metadata can support reuse and promote consistency in how scales are used and scored across studies.

2.6 Licenses

Sharing data and materials is not enough; repositories should also clearly specify the conditions under which their contents may be reused. Adding a license clarifies the permissions granted to others, including whether they may use, share, adapt, distribute, and reproduce the materials in any medium or format. For example, under a Creative Commons Attribution (CC BY) license, these activities are permitted as long as users give appropriate credit to the original author(s) and source, provide a link to the license, and indicate whether changes were made. If the repository owner has not selected a specific license, check the repository's default license. A single license may not be appropriate for every file type: code generally requires a software-specific license (e.g., MIT or GPL), whereas data and other non-software outputs can be shared under a Creative Commons license or public-domain dedication (e.g., CC BY or CC0). Creative Commons recommends against using its licenses for software because they do not address software-specific issues (see [documentation] (https://creativecommons.org/faq/#can-i-apply-a-creative-commons-license-to-software)). Before applying a license, ensure that you have the rights or permission to license the material, and consider privacy, consent, funder or institutional requirements, and third-party rights. The [Choose a License] (https://choosealicense.com/non-software/) website offers guidance to help researchers select an appropiate license for research outputs.

Include the full license text in a clearly named `LICENSE` file at the repository root. GitHub allows you to add a license when creating a repository and automatically generates the corresponding file and file text based on the chosen license. If different parts of the repository have different licenses, use separate license files or state clearly in the README and repository metadata which license applies to each type of content. Also check the repository's default license before depositing files: for example, Zenodo defaults to CC BY 4.0 for records and allows different licenses to be declared for different files, such as MIT for software and CC BY for documentation. For more information, see the [Zenodo documentation] (https://help.zenodo.org/docs/deposit/describe-records/licenses/) on licenses and rights and [GitHub's guidance] (https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/licensing-a-repository) on licensing a repository.

3 Automatic assessment of FAIR data and code

We have discussed more than 100 recommendations that researchers can follow to make their repositories machine-readable. However, researchers may not have the necessary expertise, knowledge or time to learn and implement all these recommendations. Furthermore, given the large number of recommendations, it is unrealistic to expect researchers to remember and consistently apply them all. One solution is to help researchers ensure their data and code is as FAIR as possible using automation. Metacheck (DeBruine et al., 2026) is an open-source R package that has dedicated modules to help researchers iteratively improve the FAIRness of their repositories, including data and code.

Researchers can use Metacheck either by providing a manuscript containing a link to a repository or by uploading a repository folder directly. Broadly speaking, three modules are designed to automatically check whether repositories follow many of the recommendations discussed in this work: `repo_check`, `data_check` and `code_check`. For a detailed discussion on how Metacheck can be used to assess FAIR data and code practices across a corpus of articles, including an evaluation of the accuracy of these modules see [CITATION].

The `repo_check` module identifies links to repositories in manuscripts, examines their contents, and performs checks on the repository structure and its files. One of its outputs is a

table listing all files identified in the repository, including their names, file types and paths (Table Table 3).

All Metacheck modules use a traffic-light system to indicate potential problems. For `repo_check`, warnings relate to the repository level and may include missing README files, files that do not belong to the repository, or issues with file naming. Running the module on the example repository produces the following traffic-light output:

red

A red light signals that issues have been identified and must be addressed. In addition to the traffic-light system, the module provides a summary of its findings. For example, when applied to the example repository, the module returns the following summary:

- We found 7 files in 1 repository.
- We found 1 README file and 0 repositories without READMEs.
- We left out 1 file that does not belong in a repository: operating system (1). Remove it from the repository.

The summary indicates that the repository contains an unnecessary file that should be removed. The repo_check module also provides a table listing files that may need to be removed, including their paths, type, and severity (Table Table 4):

In this example, a DS_Store file is identified. This is a macOS system file that stores folder-specific settings and is not relevant to the research project. It should therefore be removed from the repository.

The `data_check` module examines the files in a research repository to identify those containing data, such as spreadsheets and CSV files. For each data file it can read, the module provides an overview of its columns, including their names and data types, and checks for potential issues, such as entirely empty columns or inconsistencies in how categorical values are recorded. Running the module on the repository example, returns the following summary:

- We classified 7 files: 1 data, 1 code, 5 documentation.
- We found 1 tabular data file and extracted 9 columns from 1 of them.

- We grouped the files into 1 study group (ex1), plus 1 collection-level file (README/ro-crate metadata). We ran automated data-quality checks on 9 columns; 5 columns have at least one potential issue (1 column with inconsistent category casing; 1 column with no observed values (entirely empty); 1 column with numeric values stored as text; 1 column > whose name suggests personal information; 1 column whose values look like personal information; 1 column holding an SPSS > “Select Cases” filter (analyses may have used only a subset of rows)). We detected demographic columns for Age, Gender.

The module identified 5 columns might have at least one potential issue. These include inconsistent capitalisation of categorical values, an entirely empty column, numeric values stored as text, a column whose name suggests it may contain personal information, a column whose values appear to contain personal information, and a column containing an SPSS “Select Cases” filter, which may indicate that analyses were performed on only a subset of the rows.

The module also provides a summary of the potential issues identified, including the affected file, column, type of check, and details of the finding (Table Table 5).

The `code_check` module examines the code files linked to a paper and identifies common issues that may prevent the code from running elsewhere. These include hard-coded file paths pointing to the author’s computer, missing comments explaining analytical steps, and references to packages that are not explicitly loaded in the script. These checks help identify potential barriers to computational reproducibility. RunninG the module returns the following summary:

- We found 1 R, 0 Python, 0 SAS, 0 SPSS, 0 Stata, 0 Mplus, 0 MATLAB, and 0 JASP code file.
- 1 code file (analysis.qmd) had no comments.
- setwd() calls were found.
- All libraries/imports were loaded in one block.

- No pinned R/package environment (renv.lock, sessionInfo(), or groundhog/checkpoint) was found. Consider adding a solution to improve computational reproducibility.

The module identified three potential issues with the analysis code. These include the absence of comments in the analysis.qmd file, the use of setwd() to set the working directory, and the absence of a pinned R/package environment (e.g., renv.lock). The latter makes it more difficult to reconstruct the software environment required to run the analysis.

The `codebook_check` module uses the results from data_check to examine whether each data column is described in an accompanying codebook. Columns with a corresponding description are labelled, whereas those without a description are classified as unlabelled. This helps identify variables that require further documentation to make datasets easier to understand and reuse. The example repository contains a codebook file with descriptions of the dataset variables (Table Table 6), shown below:

Running the module returns the following summary:

- We parsed 2 variable definitions from codebook/README files.
- 2 of 9 data columns (22%) are documented in a codebook; 7 are not.

The module correctly identifies the seven data columns that are not documented in the codebook, highlighting where additional variable descriptions are needed.

Together, these modules illustrate how Metacheck can help researchers identify potential issues that hinder the FAIRness of research data and code, supporting efforts to make their repositories and their contents machine-readable.

Discussion

Data and code should be shared as FAIR as possible to facilitate verification, reproducibility and reuse. Repositories containing FAIR data and code should enable machines to find and access all content. Achieving this requires several practices: data and code should be made available in online repositories from which they can be downloaded; files and folders should have clear names and be organized in a consistent folder structure that facilitates their automatic classification and identification of relationships between files;

repository contents should be described using machine-readable metadata; files should be shared in open, machine-readable formats; codebooks should be included; data files should be well-structured; and, where relevant, metadata describing measures and tasks should be provided. The use of schemas such as Psych-DS, RO-Crate JSON-LD, DDI-Lifecycle, CRediT, Open Scales Definition, and Behaverse Data Model (Defossez et al., 2020) can facilitate FAIR data and code.

Adhering to best practices for FAIR data and code is complex and effortful. This document contains more than 100 recommendations and it is unrealistic to expect researchers and data managers to remember implement them all. It becomes essential to assist researchers in the FAIRification of their research outputs with automated tools, such as Metacheck (DeBruine et al., 2026). However, automated checks of whether metadata follow a particular schema cannot establish whether their contents are accurate or complete. Researchers must still verify, for example, that units are correct, processing steps are adequately described, and a codebook reflects the shared dataset. Guidance on scientific code review similarly emphasises the value of informed human examination alongside technical checks (Hillemann et al., 2025; Ivimey-Cook et al., 2023; Rokem, 2024).

The potential of machine-readable data and code extends beyond individual repositories. If data repositories can be scanned automatically, information about their contents could be processed automatically, and stored in a central database. Researchers could then search for data files containing particular scales or tasks while filtering by characteristics of the sample such as demographic information. This could greatly facilitate the reuse of research data. Machine-readable code and data could also make automated reproducibility checks possible increasing the quality of scientific manuscripts, without increasing the burden on peer reviewers. More broadly, machine-readable repositories could help researchers demonstrate adherence to research-integrity requirements to FAIR data and code when possible. They could also allow universities, funders, and other research organizations to automatically check whether shared research outputs meet relevant data-sharing requirements.

As adding metadata to repositories remains challenging, and the number of best practices in data sharing makes it difficult to remember and implement them all, automated

tools will therefore play a big role in the adoption of FAIR principles to data and code. If data and code can be processed by machines, it becomes possible not only to validate the extent to which a repository follows FAIR principles, but also to guide researchers interactively towards improving its findability, accessibility, interoperability, and reusability.

References

Abdill, R. J., Talarico, E., & Grieneisen, L. (2024). A how-to guide for code sharing in biology. PLOS Biology, 22(9), e3002815. https://doi.org/10.1371/journal.pbio.3002815

ALLEA. (2023). The european code of conduct for research integrity: Revised edition. ALLEA. https://doi.org/10.26356/ECOC

Arslan, R. C. (2019). How to automatically document data with the codebook package to facilitate data reuse. Advances in Methods and Practices in Psychological Science, 2(2), 169–187. https://doi.org/10.1177/2515245919838783

Barker, M., Chue Hong, N. P., Katz, D. S., Lamprecht, A.-L., Martinez-Ortiz, C., Psomopoulos, F., Harrow, J., Castro, L. J., Gruenpeter, M., Martinez, P. A., & Honeyman, T. (2022). Introducing the FAIR Principles for research software. Scientific Data, 9, 622. https://doi.org/10.1038/s41597-022-01710-x

Buchanan, E. M., Crain, S. E., Cunningham, A. L., Johnson, H. R., Stash, H., Papadatou-Pastou, M., Isager, P. M., Carlsson, R., & Aczel, B. (2021). Getting started creating data dictionaries: How to create a shareable data set. Advances in Methods and Practices in Psychological Science, 4(1), 2515245920928007. https://doi.org/10.1177/2515245920928007

Chue Hong, N. P., Katz, D. S., Barker, M., Lamprecht, A.-L., Martinez, C., Psomopoulos, F. E., Harrow, J., Castro, L. J., Gruenpeter, M., Martinez, P. A., Honeyman, T., Struck, A., Lee, A., Loewe, A., van Werkhoven, B., Jones, C., Garijo, D., Plomp, E., Genova, F., … RDA FAIR4RS WG. (2022). FAIR principles for research software (FAIR4RS principles). Research Data Alliance. https://doi.org/10.15497/RDA00068

DeBruine, L., Mesquida, C., Werner, J., & Lakens, D. (2026). Metacheck: Check research outputs for best practices. https://doi.org/10.5281/zenodo.20704754

Defossez, A., Ansarinia, M., Clocher, B., Schmück, E., Schrater, P., & Cardoso-Leite, P. (2020). The structure of behavioral data. arXiv:2012.12583. https://doi.org/10.48550/arXiv.2012.12583

Eglen, S. J., Marwick, B., Halchenko, Y. O., Hanke, M., Sufi, S., Gleeson, P., Silver, R. A., Davison, A. P., Lanyon, L., Abrams, M., Wachtler, T., Willshaw, D. J., Pouzat, C., &

Poline, J.-B. (2017). Toward standard practices for sharing computer code and programs in neuroscience. Nature Neuroscience, 20(6), 770–773. https://doi.org/10.1038/nn.4550

Hardwicke, T. E., Bohn, M., MacDonald, K., Hembacher, E., Nuijten, M. B., Peloquin, B. N., deMayo, B. E., Long, B., Yoon, E. J., & Frank, M. C. (2021). Analytic reproducibility in articles receiving open data badges at the journal Psychological Science: An observational study. Royal Society Open Science, 8(1), 201494. https://doi.org/10.1098/rsos.201494

Hillemann, F., Burant, J. B., Culina, A., & Vriend, S. J. G. (2025). Code review in practice: A checklist for computational reproducibility and collaborative research in ecology and evolution. EcoEvoRxiv; EcoEvoRxiv. https://doi.org/10.32942/X26S6P

Ivimey-Cook, E. R., Culina, A., Dimri, S., Grainger, M., Kar, F., Lagisz, M., Moran, N., Nakagawa, S., Roche, D., Tattan, S., Sánchez-Tójar, A., Windecker, S., & Pick, J. L. (2025). TADA! Simple guidelines to improve analytical code sharing for transparency and reproducibility. EcoEvoRxiv; EcoEvoRxiv. https://doi.org/10.32942/X2D93K

Ivimey-Cook, E. R., Pick, J. L., Bairos-Novak, K. R., Culina, A., Gould, E., Grainger, M., Marshall, B. M., Moreau, D., Paquet, M., Royauté, R., Sánchez-Tójar, A., Silva, I., & Windecker, S. M. (2023). Implementing code review in the scientific workflow: Insights from ecology and evolutionary biology. Journal of Evolutionary Biology, 36(10), 1347–1356. https://doi.org/10.1111/jeb.14230

Jenkins, G. B., Beckerman, A. P., Bellard, C., Benítez-López, A., Ellison, A. M., Foote, C. G., Hufton, A. L., Lashley, M. A., Lortie, C. J., Ma, Z., Moore, A. J., Narum, S. R., Nilsson, J., O'Boyle, B., Provete, D. B., Razgour, O., Rieseberg, L., Riginos, C., Santini, L., … Peres-Neto, P. R. (2023). Reproducibility in ecology and evolution: Minimum standards for data and code. Ecology and Evolution, 13(5), e9961. https://doi.org/10.1002/ece3.9961

Lakens, D., & DeBruine, L. M. (2021). Improving transparency, falsifiability, and rigor by making hypothesis tests machine-readable. Advances in Methods and Practices in Psychological Science, 4(2), 2515245920970949.

https://doi.org/10.1177/2515245920970949

Lamprecht, A.-L., Garcia, L., Kuzak, M., Martinez, C., Arcila, R., Martin Del Pico, E., Dominguez Del Angel, V., van de Sandt, S., Ison, J., Martinez, P. A., McQuilton, P., Valencia, A., Harrow, J., Psomopoulos, F., Gelpi, J. Ll., Chue Hong, N., Goble, C., & Capella-Gutierrez, S. (2020). Towards FAIR principles for research software. Data Science, 3(1), 37–59. https://doi.org/10.3233/DS-190026

Michener, W. K. (2015). Ten simple rules for creating a good data management plan. PLoS Computational Biology, 11(10), e1004525.

Miksa, T., Simms, S., Mietchen, D., & Jones, S. (2019). Ten principles for machine-actionable data management plans. PLoS Computational Biology, 15(3), e1006750.

Mueller, S. T., & Piper, B. J. (2014). The Psychology Experiment Building Language (PEBL) and PEBL test battery. Journal of Neuroscience Methods, 222, 250–259. https://doi.org/10.1016/j.jneumeth.2013.10.024

NWO. (2026). Netherlands Code of Conduct for Research Integrity.

Pampel, H., Weisweiler, N. L., Strecker, D., Witt, M., Vierkant, P., Elger, K., Bertelmann, R., Buys, M., Ferguson, L. M., Kindling, M., Kotarski, R., & Petras, V. (2023). Re3data – Indexing the Global Research Data Repository Landscape Since 2012. Scientific Data, 10(1), 571. https://doi.org/10.1038/s41597-023-02462-y

Pick, J. L., Allen, B. J., Bachelot, B., Bairos-Novak, K. R., Brand, J. A., Class, B., Dallas, T., D'Amelio, P. B., Fenollosa, E., Fernández-Juricic, E., Gomes, D. G. E., Grainger, M. J., Guillemaud, T., John, C., Krasnow, R., Lagisz, M., Lequime, S., Maynard, D. S., Nakagawa, S., … Ivimey-Cook, E. R. (2026). The SORTEE guidelines for data and code quality control in ecology and evolutionary biology. Peer Community Journal, 6, e20. https://doi.org/10.24072/pcjournal.687

Rokem, A. (2024). Ten simple rules for scientific code review. PLOS Computational Biology, 20(9), e1012375. https://doi.org/10.1371/journal.pcbi.1012375

Soiland-Reyes, S., Sefton, P., Crosas, M., Castro, L. J., Coppens, F., Fernández, J. M., Garijo, D., Grüning, B., La Rosa, M., Leo, S., Ó Carragáin, E., Portier, M., Trisovic, A., RO-Crate Community, Groth, P., & Goble, C. (2022). Packaging research artefacts

with RO-Crate. Data Science, 5(2), 97–138. https://doi.org/10.3233/DS-210053

Trisovic, A., Lau, M. K., Pasquier, T., & Crosas, M. (2022). A large-scale study on research code quality and execution. Scientific Data, 9, 60. https://doi.org/10.1038/s41597-022-01143-6

Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., Blomberg, N., Boiten, J.-W., da Silva Santos, L. B., Bourne, P. E., Bouwman, J., Brookes, A. J., Clark, T., Crosas, M., Dillo, I., Dumon, O., Edmunds, S., Evelo, C. T., Finkers, R., … Mons, B. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3, 160018. https://doi.org/10.1038/sdata.2016.18

Table 1
Comparison of selected research repositories

Repository	Main advantages	Limitations and points to check
Zenodo	DOIs, versioning, API access, and options to restrict access to files.	No folder hierarchy. Metadata remain publicly accessible even when files are restricted. Changes to published files require a new version.
GitHub	Code development and collaboration, version history, releases, and API access. Public releases can be archived through Zenodo and assigned a DOI.	Primarily designed for code development and collaboration rather than long-term data archiving. Zenodo DOI integration requires a public repository.
Open Science Framework (OSF)	Project collaboration and view-only links for sharing materials during peer review.	No new projects or components from November 16, 2026; existing projects become read-only on February 19, 2027.
ResearchBox	Search functionality for data and code, automatically generated codebooks, and a documented API.	Check whether its preservation arrangements, access options, and file-size limits meet the project's requirements. (API)
Figshare	DOIs, versioning, API, and support for a wide range of research outputs.	Storage limits and available features may differ between Figshare.com and institutional Figshare services.
PsychArchives	Psychology-focused repository that links related research outputs and provides anonymized reviewer links.	Primarily intended for psychology and related disciplines. Datasets must be anonymized. The standard file-size limit is 1 GB, although larger deposits can be requested. Check API documentation and options for automated access to restricted files.
DataverseNL	Dataset DOIs, versioning, metadata, access management, and API-based upload.	Institutional policies and available support may differ.
UK Data Service ReShare	Social-science repository with review of deposits and open or safeguarded access.	Deposits must meet the UK Data Service's scope and policies. Access to safeguarded files may require registration or permission.

Table 2
Example of codebook

variable-name	label	values	missing-values	question	coding-instructions
pid	Participant identifier				
age	Age in years		-99	What is your age in years?	
gender	Self-reported gender	1 = Man; 2 = Woman; 3 = Non-binary	99	Which best describes your gender?	
bds_1	Breakup Distress item 1	1 (Strongly disagree) to 5 (Strongly agree)	99	I feel overwhelmed by the breakup.	
bds_3	Breakup Distress item 3	1 (Strongly disagree) to 5 (Strongly agree)	99	I am coping well.	Reverse scored: 6 - bds_3
bds_mean	Breakup Distress mean score				Mean of bds_1 to bds_10 after reversing bds_3

Table 3
Files identified in the repository by the `repo_check` *module.*

Name	Type	Path
analysis.pdf	text	analysis.pdf
analysis.qmd	code	analysis.qmd
codebook.csv	codebook	codebook.csv
LICENSE		LICENSE
README.md	readme	README.md
references.bib		references.bib
stroop.csv	data	stroop.csv

Table 4
Unnecessary files identified in the repository by the `repo_check` module.

Path	Kind	Severity
.DS_Store	operating system	problem

Table 5
Potential issues identified in the repository by the `data_check` module.

Name	Column	Check	Detail
stroop.csv	Gender	Case issues	Categories differing only by case: Female/female
stroop.csv	Age	Numeric as text	Column is 99% numeric but 1 value cannot be parsed: ”unknown”
stroop.csv	Email	Personal info (values)	Values look like personal information: email (132 values, 100%). Review before sharing.
stroop.csv	Email	Personal info (column name)	Column name suggests personal information (matched: email). Review before sharing.
stroop.csv	filter_$	SPSS filter variable	SPSS ”Select Cases” filter variable: every row is selected (value 1), so the file appears to have been saved after deleting unselected cases – the shared data are a pre-filtered subset of what was collected.
stroop.csv	Notes	Empty column	Column is empty: all 132 values are missing.

Table 6
Variables described in `codebook.csv`.

Variables	Label	Source
Congruent	Reaction time in congruent condition	codebook.csv
Incongruent	Reaction time in incongruent condition	codebook.csv